



DEEP LEARNING

AULA 3



Prof. Eng. Gian Carlo Brustolin



CONVERSA INICIAL

Quando estudamos redes neurais artificiais, após conhecermos sua estrutura básica e funcionamento, naturalmente voltamos nossa curiosidade para o processo de aprendizado destas estruturas matemáticas capazes de pensar.

O processo de treinamento de uma rede neural artificial, de forma semelhante a alguns algoritmos de *machine learning* (ML), pode ter uma aproximação supervisionada ou não supervisionada. No primeiro caso, treinamos o modelo a partir de pares de vetores conhecidos de entrada e saída, criando uma memória na rede pela alteração de suas sinapses. Este processo nos parece lógico e intuitivamente compreensível.

O treinamento não supervisionado, por sua vez, tem sua compreensão um pouco menos intuitiva. Esta aproximação de aprendizagem presume que a rede neural artificial possui uma capacidade nativa para encontrar padrões no vetor de entrada. Para que possamos compreender este processo e os algoritmos que permitem tal treinamento, será necessário estudar a teoria estatística do aprendizado, que nos fornecerá os conceitos necessários.

No caso de redes neurais artificiais profundas, não apenas as aproximações supervisionada e não supervisionada estrita são viáveis como métodos de treinamento. Uma terceira aproximação, por vezes incluída na categoria de treinamento não supervisionado, é bastante usual, principalmente em aplicações voltadas à robótica e a problemas de decisão estocástica. Trata-se da aprendizagem por reforço (*Reinforcement Learning* – RL).

Neste capítulo, estudaremos a teoria estatística da aprendizagem voltada a redes neurais artificiais profundas e apresentaremos algoritmos de treinamento não supervisionado e por reforço aplicáveis a estas redes.

Ao final deste capítulo, você completará o ferramental teórico, construído até aqui, necessário para o estudo das RNAs profundas aplicadas em DL.

Vamos dar início a este empolgante estudo.

TEMA 1 – CONCEITOS DE TREINAMENTO NEURAL



O uso de RNAs para resolução de problemas é bastante *sui generis*, uma vez que a modelagem matemática se restringe aos neurônios, e não ao problema em si. Para que particularizemos a RNA em direção ao problema necessitaremos “ensiná-la”. Este treinamento da rede exigirá, por sua vez, algoritmos que simulam, em boa parte, o processo de construção do conhecimento humano.

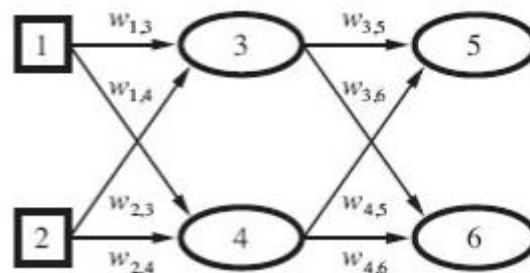
Vamos, então, estudar os processos de aprendizagem e memória de redes de neurônios artificiais, que resultarão na adaptação de uma RNA genérica para a solução de um problema específico.

1.1 A Matriz W

Antes de iniciarmos a discussão do treinamento de uma RNA, é importante que você revise os conceitos da modelagem matemática do neurônio artificial, bem como a estrutura básica de uma RNA.

Utilizaremos, no primeiro momento, como modelo para ilustrar o processo de treinamento de uma RNA, uma MLP rasa, conforme ilustrado a seguir.

Figura 1 – *Multilayer Perceptron* - MLP



Fonte: Norvig, 2013, p. 637.

O treinamento de uma rede neural se dá pelo ajuste de seus parâmetros livres (pesos sinápticos e bias). Desta forma, definida uma arquitetura para a RNA, seus parâmetros livres conterão valores aleatórios, ou propositalmente, estes serão assim inicializados.

A literatura recomenda, no caso da inicialização forçada, que especialmente os valores dos pesos sinápticos sejam inicializados com valores no intervalo de -1 e 1. Quanto ao bias, embora seja considerado parâmetro livre, seu valor normalmente permanece unitário. A alteração deste valor pode ocorrer na inicialização da rede, para ajustar o ponto de decisão da função de ativação.



Esta alteração pode ser convenientemente decidida para a camada de entrada e, em casos mais raros, para a camada de saída.

O ajuste do bias dos neurônios ocultos é, de maneira geral, evitado, uma vez que não é possível, para estes neurônios, seguir critérios lógicos de modificação. Discutiremos este assunto um pouco mais adiante em nossa caminhada, quando abordarmos algoritmos avançados de treinamento. Por hora consideraremos o valor do bias, exceto se afirmarmos, de outro modo, unitário e inalterado durante o processo de treinamento.

Feitas estas observações, podemos dizer que uma RNA possui uma “memória associativa” da tarefa, distribuída nos pesos sinápticos. Haykin (2011, p. 101) afirma que os pesos de uma RNA criam um mapeamento distribuído entre os vetores de entrada e o vetor de saída.

Desta forma, considerando a RNA como um operador vetorial e denominando de:

Y o vetor que contém os valores de saída de todos os neurônios da camada exposta;

X o vetor que contém os valores de entrada, que produzem a saída **Y** e

W a matriz ou vetor multidimensional dos pesos sinápticos.

Podemos escrever:

$$Y = W * X$$

Para que esta equação vetorial seja válida, devemos supor a dimensionalidade de **W** determinada pelo vetor **X**, ainda precisaremos fazer a dimensão do vetor **Y**, a mesma de **X**.

Como já afirmamos, o treinamento é o ajuste de **W** para que a rede responda como esperado. No treinamento supervisionado conhecemos a saída da rede. Neste caso, se chamarmos o vetor de saída desejado da RNA de **D**, treinar será fazer $Y \rightarrow D$ pelo ajuste de **W** (Norvig, 2013, p. 607).

No treinamento não supervisionado e por reforço, esta inequação não é válida, já que inexiste **D**. Vamos tratar destes outros dois tipos de treinamento em momentos posteriores, ainda nesta etapa.

Em qualquer um dos casos, a matriz ‘W’ pode ser entendida como a **memória da rede**. Esta memória é treinada, de forma recursiva, para absorver o conhecimento que particularizará a RNA para a solução do problema que estamos enfrentando.



1.2 O Vetor X

A entrada de uma RNA é representada pelo vetor X . Este vetor será composto pelas “ i ” variáveis independentes de entrada $X = \{x_1, x_2, \dots, x_i\}$. Dito de outra forma, por x_i ser uma variável, poderá assumir infinitos valores. Uma RNA treinada, ao receber um determinado vetor X_j , deverá apresentar na saída o vetor D_j . Antes do treinamento, ao receber X_j , a saída será Y_j . Y_j poderá ou não coincidir com D_j , na absoluta maioria dos casos diferirá.

Os valores de X_j poderão, hipoteticamente, assumir infinitos valores válidos, estes valores corresponderão ao comportamento válido das variáveis x_i .

Um pouco confuso?

Vamos tentar um exemplo:

Suponha uma RNA que deve decidir sobre o momento de corte de uma árvore. Para tanto, temos disponíveis três variáveis de entrada, espessura do tronco, altura e espessura da casca. Suponha que valores válidos para a espessura do tronco sejam valores entre 25 e 78 cm; já a altura variará entre 100 e 250 cm; ao passo que a espessura da casca excursionará entre 2 e 4,5 cm.

Com estas considerações, podemos ter os seguintes vetores válidos X :

$X_1 = \{30; 136; 2,23\}$

$X_2 = \{39; 176; 3,23\}$

$X_3 = \{60; 236; 3,03\}$

Os valores de X_i , conhecidos, que já ocorreram em alguma medição que se fez das variáveis de entrada, são chamados amostras ou padrões.

1.3 Padrões de treinamento de uma rede neural artificial

Já sabemos que, no modelo supervisionado, treinar uma RNA significa fazer $Y \rightarrow D$ pelo ajuste de W . Este ajuste será feito com o uso dos padrões X que definimos anteriormente. Dito de outra forma, passaremos os padrões conhecidos $X_1, X_2, \dots, X_n$, pela rede neural, obtendo as saídas vetoriais $Y_1, Y_2 \dots Y_n$.

Para que ajustemos os valores de W , precisamos conhecer $D_1, D_2, \dots, D_n$ (valores desejados de saída da RNA).

Desta forma, em um treinamento supervisionado, os padrões de treinamento não serão compostos apenas por X_i , mas também por D_i , assim



teremos como padrões os pares biunívocos de vetores $P1=[X1, D1]$; $P2=[X2, D2]$; ...; $Pi=[Xi, Di]$.

Voltemos a nosso exemplo do corte de árvores, para treinamos a RNA, devemos conhecer, para os vetores Xi , qual a decisão tomada em eventos anteriores, se cortamos ou não a árvore, que será a saída desejada Di . Assim, os padrões do exemplo poderiam ser:

$X1=\{30; 136; 2,23\}$; $D1= 0$ (não cortar)

$X2=\{39; 176; 3,23\}$; $D2 = 1$ (cortar)

$X3=\{60; 236; 3,03\}$; $D2 = 1$ (cortar)

Agora, basta-nos ajustar os valores dos diversos wi para que, $Yi \rightarrow Di$. Há vários métodos para ajustar wi , o mais usual é a retropropagação de erro. Neste método, calcula-se o erro de saída pela equação:

$$e_i = D_i - Y_i$$

O valor e_i será, então, usado, recursivamente, como parâmetro para ajuste de wi .

1.4 Generalização

Ainda não sabemos como efetivamente treinamos uma RNA, mas já sabemos o que é o treinamento. Treinar a RNA se quer que esta ganhe a possibilidade de generalização.

Por generalização entendemos a capacidade de prever qual a saída desejada de uma entrada qualquer, mesmo que diferente de P . Dito de outra forma, se apresentarmos à rede um novo vetor Xt , cujo valor Dt é desconhecido, como o treinamento fez $Y \rightarrow D$, podemos concluir que Yt aproximará convenientemente Dt .

Voltando ao exemplo da árvore, se oferecermos à rede treinada um novo valor de Xt , diferente dos padrões $X1$, $X2$ e $X3$, a RNA será capaz de generalizar, decidindo, com boa possibilidade de acerto, pelo corte ou não da árvore.

Do que acabamos de aceitar depreende-se que os padrões de treino são amostras estatísticas de um determinado fenômeno e, como tal, tendem a representar o fenômeno com boa aproximação. Desta forma, o aprendizado da rede é um fenômeno estocástico.



Assim, a generalização é, de fato, uma probabilidade, eventualmente calculável, da rede mapear corretamente a saída desejada. Vamos examinar este tópico com um pouco mais de detalhes a seguir.

TEMA 2 – TEORIA ESTATÍSTICA DO APRENDIZADO

Acabamos de concluir que o processo de treinamento de uma RNA não é determinístico, ou seja, estará sujeito a erros, tanto na fase de treinamento (erro de treino) quanto na fase de operação (erro de generalização). Estes erros são calculáveis ou, ao menos, estimáveis. O controle dos erros de treino e generalização são realizados pela teoria estatística do aprendizado.

2.1 Erro de treino

Nosso estudo anterior demonstrou que ao oferecermos um conjunto de padrões de treinamento para uma RNA e corrigirmos os parâmetros livres, de forma iterativa, esta rede “memorizará” um mapeamento entre entrada e saída.

No caso do treinamento supervisionado, a relação matemática entre X e D não precisa ser conhecida para que possamos “ensinar” a rede a reproduzir, ao menos com boa probabilidade de acerto, D em Y . Dito de outra forma, para maior clareza, devemos apenas conhecer os padrões $[X, D]$, mas não, necessariamente, a equação matemática que relaciona X e D . O treinamento fará $Y \rightarrow D$, e a rede, então, agirá como se conhecesse a relação matemática.

Você deve estar se perguntando, neste ponto, por que D não é igual a Y ? Este é um questionamento bastante válido. Pense que, quando calculamos o erro de estimativa (e_i), apresentado em (1.3), o fizemos em relação ao padrão “ i ”, ou seja, quando ajustarmos o peso sináptico o faremos em relação a este mesmo padrão. Posteriormente, ao analisarmos o padrão “ $i+1$ ”, haverá um novo ajuste do mesmo peso sináptico.

Desta forma, após vários ajustes, um peso sináptico qualquer, pertencente à rede, terá um valor que “resume” todos os ajustes feitos, mas que **não representa o ajuste perfeito de nenhum dos padrões**.

Por este motivo $Y \rightarrow D$, mas, essencialmente, $Y \neq D$. Desta forma, poderíamos escrever:

$$D = Y + \varepsilon$$



Onde $\mathcal{E}$ é o erro na estimativa de $\mathbf{D}$ por $\mathbf{Y}$ durante o treinamento. Há várias formas de se estimar $\mathcal{E}$ a partir dos erros $\mathbf{e}_i$, referentes a cada padrão de treino “ i ”, a mais frequente é considerar (Haykin, 2011, p. 110):

$$\mathcal{E} = \frac{1}{2} \sqrt{\sum (\mathbf{e}_i)^2}$$

Como o treinamento é um processo de aproximação $\mathbf{Y} \rightarrow \mathbf{D}$, espera-se que $\mathcal{E}$ tenda a zero para que $\mathbf{Y} \approx \mathbf{D}$. Aliás, esta consideração gera a aproximação mais frequente para o treinamento neural supervisionado, a retropropagação de erro. Neste processo forçamos, no ajuste dos pesos, a relação com o erro resultante da propagação de $\mathbf{X}$. Não exploraremos o método de retropropagação nesta etapa, basta-nos, por hora, aceitar que, considerando a necessidade de redução do erro de estimativa, podemos corrigir um peso sináptico de forma iterativa, como segue (Haykin, 2013, p. 193):

$$\mathbf{w}i = \mathbf{w}i + \Delta \mathbf{w}i \quad \text{onde: } \Delta \mathbf{w}i = \delta \mathbf{x}i * \varphi'(\mathbf{v}) * \mathcal{E}$$

Sendo δ velocidade de aprendizagem e $\varphi'(\mathbf{v})$ a derivada da função de ativação.

2.2 Erro de generalização

Sabemos, então, dado ao processo de ajuste de $\mathbf{W}$ por vários padrões diferentes, que $\mathbf{Y} \approx \mathbf{D}$, mas é impossível fazermos $\mathbf{Y} = \mathbf{D}$. Para a próxima análise imagine que, após concluído o treinamento, $\mathbf{Y} = \mathbf{D} + \mathcal{E}$, como, para este treinamento, escolhemos apenas alguns padrões (ou instâncias) do espaço amostral, não há garantia de que, quando a rede entrar em modo de generalização, o erro de generalização seja idêntico a $\mathcal{E}$.

Chamaremos ao erro que poderá ocorrer durante a generalização de $\mathcal{E}_g$, assim, quando a RNA entrar em operação (Haykin, 2011, p. 115):

$$\mathbf{Y} = \mathbf{D} + \mathcal{E}_g$$

Se escolhermos os padrões de treino de forma estatisticamente correta, respeitando as recomendações de amostragem $\mathcal{E}_g \rightarrow \mathcal{E}$, mas pelo mesmo motivo estatístico, $\mathcal{E}_g \neq \mathcal{E}$. Estas conclusões tornam o processo de aprendizagem fortemente estocástico e a presença de erro um fenômeno natural em redes neurais. O erro, embora inevitável, será sempre estatisticamente controlável.



2.3 Escolha dos padrões

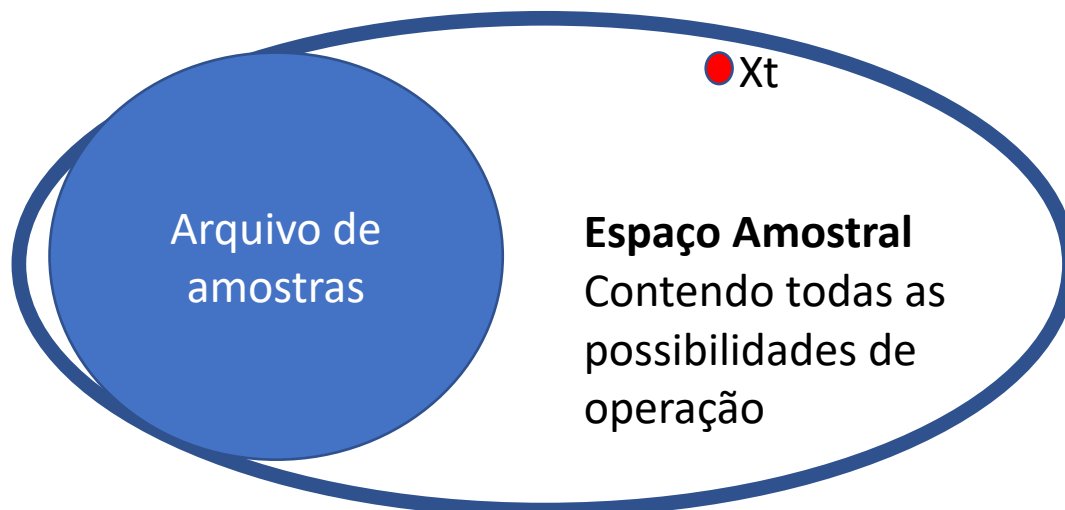
Se você estudou algoritmos de ML, está habituado a treinar um modelo a partir de uma massa de dados. Normalmente, nesta aproximação, temos um arquivo contendo uma boa quantidade de amostras, após depurarmos estas amostras, eliminando ou suprimindo aquelas incompletas, segregamos os dados em dois conjuntos, de treino e de teste.

Esta segregação é feita por um processo aleatório, reservando em torno de 80% das amostras para treino (Gerón, 2021). De maneira geral, esta separação é deixada a cargo de operações pré-programadas em bibliotecas. A seguir um exemplo desta divisão, em Python, com uso da biblioteca sklearn:

```
Variável = train_text_split(x.y.test_size=0.20,random_state=40)
```

Observe que não controlamos a escolha das amostras de teste, tampouco de treino. Por ser um processo aleatório (randômico) há uma boa probabilidade de não ocorrer vício nesta seleção. Em casos críticos, entretanto, esta presunção pode não ser verdadeira. Imagine o diagrama de Venn a seguir:

Figura 2 – Amostragem enviesada



Fonte: Brustolin, 2022.

Neste exemplo, o arquivo de amostras contém padrões de uma área determinada do espaço amostral total, dito, também, espaço de hipóteses. A bipartição aleatória fará com que as estimativas de erro, obtidas a partir do conjunto de testes, não represente, convenientemente, o erro de generalização. O erro de estimativa de X_t será, seguramente, maior que o calculado como base no conjunto “aleatório” de testes.



Desta forma, ao tomarmos um conjunto de amostras, mesmo antes do pré-tratamento, é necessário verificar a sua representatividade estatística. Costuma-se observar, se conhecida a excursão máxima de cada componente de $\mathbf{X}$, a variância das amostras em relação a ela. Esta avaliação, por vezes, nos obrigará a desprezar parte das amostras, de forma a não induzir um mapeamento enviesado de apenas uma parte do espaço amostral.

Dito de outra forma, quanto mais padrões de uma determinada região do espaço de hipóteses, maior será a especialidade da RNA nesta região, reduzindo o ϵ_g para padrões de teste semelhantes, mas padrões de teste genéricos $\mathbf{X}_t$, estranhos a esta região, como ilustrado na figura, sofrerão com o enviesamento. Como resultado, teremos dois valores para ϵ_g . Se não agirmos sobre a variância dos padrões, o ϵ_g fora da área de amostragem será incontrolável.

Este cuidado normalmente é excessivo em algoritmos de ML, mas quando utilizamos redes neurais profundas para suportar a extração de parâmetros em direção a métodos de RL o tema ganha importância. Outro campo de estudos no qual a análise de desenviesamento é importante é no processo de treinamento não supervisionado. Em breve esta afirmação se tornará evidente.

TEMA 3 – TREINAMENTO NÃO SUPERVISIONADO

Até este momento, analisamos o treinamento neural com exemplos de métodos supervisionados. Assim o fizemos por serem estes de maior facilidade para a construção do conhecimento sobre aprendizado em RNAs.

Vamos agora entender como uma RNA, e em especial aquelas profundas, podem aprender sem que nada se saiba sobre a saída que desejamos da rede.

3.1 Natureza e padrões naturais

A questão da existência preponderante de padrões na natureza, chamada de princípio ontológico genérico, já gerou infinitas discussões no campo científico. O principal motivo é que qualquer regra classificatória terá exceções e, por esta razão, de fato, a regra seria incompleta ou falsa.

Na verdade, exceções não invalidam a regra genérica, mas acrescentam complexidade, às vezes intransponível, para sua modelagem precisa. Norvig (2013, p. 382) afirma, a respeito do desejo por uma ontologia precisa dos



fenômenos, que “a habilidade de tratar exceções é extremamente importante, mas é ortogonal à tarefa de compreender a ontologia geral”.

Na mesma linha deste autor, pode-se afirmar que para a categorização dos fenômenos é essencial a formação do conhecimento sobre eles. Muito provavelmente por este motivo a mente humana, em sua evolução em direção à consciência e ao conhecimento, criou mecanismos instintivos de classificação. Dito de outra forma, as redes neurais biológicas tendem a buscar padrões ou classificações para entender os fenômenos do entorno.

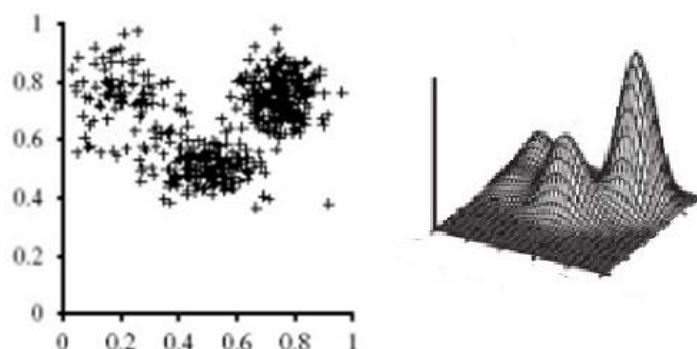
Esta característica classificatória do conhecimento inspirou Alan Kay na criação do paradigma de programação voltado a objetos. A linguagem de programação Java, ainda bastante usada, bem como o Python, são expressões computacionais de sucesso deste paradigma.

Por outro lado, se o humano, sucesso evolucionário, desenvolveu um cérebro classificatório, pode-se imaginar que a natureza que nos cerca igualmente os possua. Caso contrário, mentes caóticas teriam mais sucesso que a mente lógica humana.

Di Biasi (2013) afirma que caos dos processos naturais é apenas aparente e que há uma tendência à organização em padrões. Esta hipótese foi comprovada, em 1976, por Prigogine, observando fenômenos químicos que apresentam tendência natural à auto-organização.

A figura a seguir demonstra que, ao plotarmos amostras bivariadas e multivariadas em quantidade suficiente, regiões de concentração passam a ser percebidas.

Figura 3 – Exemplos de concentração natural de amostras bivariada e multivariada



Fonte: elaborado com base em Norvig, 2013, p. 710 e 713.



Ao aceitarmos esta tendência à redução dos fenômenos em padrões e retornando às nossas redes neurais artificiais, que são meras modelagens de nossa rede neural biológica, podemos pensar que as RNAs herdaram a tendência natural a classificar fenômenos das suas “mães biológicas”.

Haykin (2011, p. 430) comenta que “interações locais, originalmente aleatórias, entre neurônios vizinhos de uma rede, podem se fundir em estados de ordem global e finalmente, levar a um comportamento coerente na forma de padrões”.

3.2 Ambiente determinístico e episódico

O treinamento não supervisionado, também chamado auto-organizado, presume que o espaço de hipóteses seja estável, ou seja, determinístico ou ao menos episódico.

Ambientes determinísticos são meios previsíveis nos quais a amostragem dos eventos é atemporal, ou seja, coletadas amostras de eventos aleatórios não haverá necessidade de considerações temporais. Naturalmente nenhum ambiente é plenamente determinístico, mas ambientes suficientemente estáveis no tempo, para que se possa realizar amostragem de longa validade, estes serão tratados como se determinísticos fossem.

Ambientes de decisão episódica ou instantânea são aqueles nos quais o problema apresentado ao algoritmo é resolvido por uma única decisão. Desta forma, mesmo que o ambiente evolua no tempo, a amostragem é boa o suficiente para a decisão.

Quando enfrentamos problemas estocásticos ou sequenciais os algoritmos de treinamento não supervisionados terão validade relativa.

3.3 DL e aprendizagem auto-organizada

O treinamento não supervisionado, como já entendemos, é capaz de potencializar a natureza classificatória de uma RNA. Dito de outra forma, a aprendizagem auto-organizada permite que se treine uma rede para extrair, de uma amostragem, as classificações nela dormentes, de forma relativamente autônoma.

Observando esta afirmação e relacionando-a com o que afirmamos anteriormente sobre a construção do conhecimento com base na classificação



dos fenômenos, nos parecerá que o treinamento não supervisionado está relacionado com a possibilidade de extração de significado de uma amostragem.

Por outro lado, uma das mais importantes aplicações de *Deep Learning* é justamente a capacidade de extrair características (*feature learning*) ou representações (*representation learning*) de um conjunto de dados sem a interferência humana, ou seja, sem supervisor.

Desta forma, uma conclusão possível é que métodos não supervisionados estão correlacionados com o uso de redes neurais profundas para extração de significado de maneira autônoma.

TEMA 4 – ALGORITMOS DE TREINAMENTO NÃO SUPERVISIONADO

Uma vez que entendemos o fenômeno da auto-organização e sua importância para o DL, podemos estar curiosos sobre as técnicas capazes de permitir às RNAs organizarem os padrões de entrada em categorias naturais, sem a necessidade de conhecermos previamente tais categorias. Há vários algoritmos não supervisionados, e os mais tradicionais serão agora conhecidos por você.

4.1 Sinapse Hebbiana e Hipótese da Covariância

Como você já sabe, as pesquisas iniciais em torno de IA buscavam reproduzir processos biológicos ligados ao cérebro humano. O neuropsicólogo Hebb, em 1949, estudou um processo de aprendizagem natural, que envolvia o crescimento celular neural para facilitar a comunicação entre neurônios frequentemente envolvidos em um mesmo processo de interpretação de estímulos externos. Esta modificação, na arquitetura de regiões do cérebro, resultava na determinação ou modificação de comportamentos futuros.

O estudo de Hebb desvenda um método de aprendizagem, sem supervisão, baseado em alterações da intensidade da conexão sináptica, ou, se adaptarmos a conclusão para os elementos de uma RNA, baseado na majoração seletiva dos pesos sinápticos envolvidos em uma ativação frequente e contemporânea entre dois neurônios (Haykin, 2011, p. 80).

Esta modificação seletiva recebeu o nome de sinapse hebbiana. A aplicação deste método depende da apreciação ou reforço dos pesos sinápticos



que conectam neurônios, os quais sofrem ativação simultânea quando estimulados por um padrão de treinamento.

Do ponto de vista prático, a correção dos pesos sinápticos pode ser representada pela equação proposta por Haykin (2011, p.82).

$$\Delta w_{kj}(n) = \eta y_k(n) x_j(n)$$

Nesta equação, a variação do peso depende do estímulo pré-sináptico (x_j) e do resultado a este estímulo (y_k), pós-sináptico. Naturalmente esta aproximação, ao longo do tempo, levaria a saturação da sinapse, pela variação reiteradamente positiva (ou negativa). Do ponto de vista biológico, há uma limitação física desta aproximação celular, mas nos algoritmos computacionais o problema tende a persistir.

Para evitar esta saturação, em aplicações computacionais prefere-se amortizar o reforço sináptico pela covariância dos estímulos e resultados como apresentado a seguir.

$$\Delta w_{kj}(n) = \eta (x_j - \bar{x})(y_k - \bar{y})$$

Nesta nova formulação, reduzimos o valor dos estímulos e respostas pela média destes.

4.2 Aprendizado competitivo

Na aprendizagem hebbiana, vários neurônios de saída da RNA podem estar ativos simultaneamente. Dependendo da arquitetura da rede e, principalmente, das funções de ativação destes últimos neurônios, aplicações que exijam classificação estrita de padrões não serão possíveis. O algoritmo de aprendizagem competitiva atende a estas aplicações.

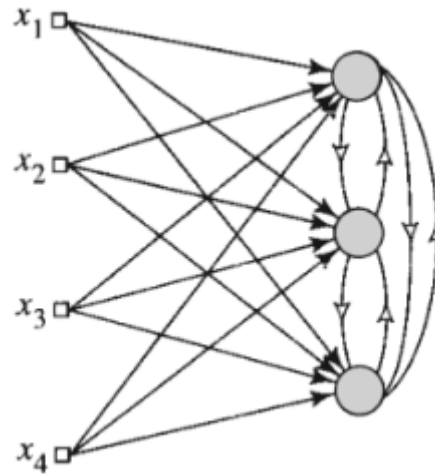
Nesta técnica, será implementado um mecanismo que permitirá aos neurônios de saída competirem pela ativação exclusiva diante de um determinado subconjunto de entradas. Desta forma, os neurônios da rede se especializam na detecção de características de classes de saída.

Este mecanismo é implementado por uma alteração arquitetural na rede, conforme ilustrado a seguir. Sempre que um neurônio da camada de saída for mais fortemente ativado, inibirá os demais. Isto se dá pela comparação dos



campos locais induzidos de cada um dos neurônios de saída. Será ativado aquele que possuir o maior campo.

Figura 4 – Aprendizagem competitiva



Fonte: Haykin, 2011, p. 84.

Os neurônios da rede envolvidos na seleção do vencedor terão seus pesos sinápticos reforçados. Este reforço pode ser obtido pela equação a seguir (Haykin, 2011, p. 84):

$$\Delta w_{kj} = \begin{cases} \eta(x_j - w_{kj}) & \text{se o neurônio } k \text{ vencer a competição} \\ 0 & \text{se o neurônio } k \text{ perder a competição} \end{cases}$$

Algoritmos competitivos, entretanto, exigem que o espaço amostral apresente alto contraste, ou seja, que as categorias naturais constituam agrupamentos suficientemente distintos, caso contrário, o algoritmo divergirá.

4.3 Clusterização

As técnicas que apresentamos até o momento são bastante clássicas e provavelmente você dificilmente ouvirá falar diretamente delas. Isto ocorre porque tais técnicas fazem parte do núcleo de algoritmos já padronizados em bibliotecas de IA, a exemplo da famosa sklearn.

Estes algoritmos padronizados realizam classificação não supervisionada e são bastante úteis quando não sabemos exatamente o que buscar na amostra ou quando não possuímos rótulos conhecidos.



Vamos dar um exemplo. Imagine que você tenha em mãos uma boa quantidade de imagens de vegetais de uma dada região, obtidas em épocas diferentes. Se desejarmos detectar quais espécies vegetais mais se desenvolveram durante os anos, estaremos diante de um problema de clusterização, ou seja, precisamos encontrar as classes de instâncias presentes.

Dito de outra forma, necessitamos, sem o auxílio de um especialista em botânica, identificar nas fotos, vegetais semelhantes que dominaram a flora daquela região. Esta é uma análise importante para enfrentamento de questões ligadas ao levantamento de impactos ecológicos de ações antrópicas, por exemplo.

Neste caso, não há rótulos prévios das fotos e técnicas supervisionadas fracassarão. O mesmo tipo de enfrentamento pode ser usado para detecção de padrões de compra de clientes, mesmo sem conhecermos antecipadamente quais padrões são estes. Uma vez identificados os clusters, ou regiões de concentração das instâncias, podemos usar técnicas supervisionadas para a análise da amostra, que agora possuirá rótulos.

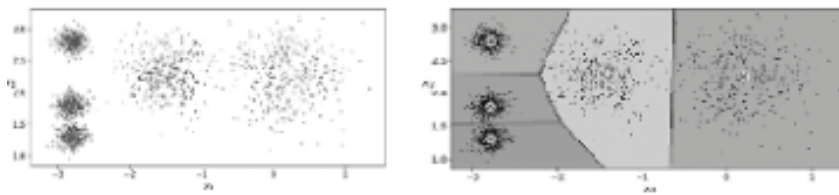
São exemplos destes algoritmos padronizados de clusterização: K-Means e DBScan. No primeiro caso, o algoritmo busca encontrar o centro do cluster e então delimitar as instâncias que participariam deste cluster, rotulando-as. O problema deste algoritmo é que precisamos fornecer o número de clusters (fator k). Apenas para ilustrar, o trecho de código a seguir faria esta clusterização de um conjunto de dados (Géron, 2021, p. 159):

```
from sklearn.cluster import KMeans
k = 5
kmeans = KMeans(n_clusters=k)
y_pred = kmeans.fit_predict(X)
```

A figura a seguir demonstra as fronteiras entre clusters de uma amostra hipotética com $k=5$. Observe que é possível que algumas amostras mais distantes do centro do cluster sejam erroneamente classificadas.



Figura 5 – Amostras não rotuladas e após a clusterização



Fonte: Géron, 2021, p. 159.

Além da clusterização, algoritmos não supervisionados são bons detectores de anomalias (*outliers*). A detecção de anomalias tem aplicação bastante importante em segurança da informação como forma de predição e localização de invasão.

TEMA 5 – TREINAMENTO POR REFORÇO

Nesta nossa caminhada sobre DL, não será nosso objetivo abordar, com profundidade, o treinamento por reforço, ou *reinforcement learning*. As redes neurais artificiais profundas, são, entretanto, uma ferramenta bastante usada como extratora de características, preparando o dataset para os algoritmos de RL. Faremos, então, uma revisão rápida sobre o tema de RL para nos focarmos na aplicação de DL como ferramenta para RL.

5.1 Introdução ao RL

O aprendizado por reforço pode ser entendido como um tipo de treinamento não supervisionado, embora normalmente seja considerado uma particularização deste.

Como já sabemos, o aprendizado não supervisionado utiliza apenas a tendência natural à auto-organização dos fenômenos como forma de treinar uma RNA classificadora. No aprendizado por reforço, o aprendizado se dá pela análise dos retornos que o meio fornece ao agente de aprendizado. Assim, embora não tenhamos uma coleção prévia de padrões de treinamento, estes padrões serão conhecidos pelo agente em sua interação com o meio, durante a operação do agente, na forma de reações do ambiente para com o agente.

O agente interpretará a reação do meio às suas ações segundo uma ótica binária. Se a reação for positiva, será dita recompensa ou reforço e poderá ser



repetida. Já uma reação negativa do meio (punição) servirá de inibidor de comportamentos semelhantes.

Quando estudamos a característica do meio para o qual a aprendizagem supervisionada foi inicialmente criada, afirmamos que este aprendizado pressupõe um ambiente estável temporalmente ou, se este não for o caso, ao menos episódico. Se você parar para pensar, entretanto, logo concluirá que os meios determinísticos, estáveis, são um caso particular dos meios instáveis ou estocásticos, que são o caso geral. Da mesma forma que, na interação com o meio, a decisão episódica é uma exceção, normalmente as decisões são sequenciais, encadeadas.

Esta conclusão, por outro lado, não invalida o estudo encetado até o momento, embora o meio estocástico e sequencial seja o caso geral, boa parte dos problemas que enfrentamos podem ser modelados de forma determinística ou episódica. O enfrentamento deste caso geral é, naturalmente, mais complexo e envolve técnicas de *reinforcement learning*.

Em um ambiente estocástico, a amostragem atemporal não é útil como forma de previsibilidade, já que não considera a evolução temporal do meio. Nesta situação, deve-se reavaliar o meio a cada decisão tomada, uma vez que a própria ação sobre o meio, propalada por nós, modifica este. As decisões tornam-se, então, por este motivo, sequenciais e interdependentes. A modelagem matemática para enfrentamento de meios com estas características foi iniciada pelo matemático russo Andrei Markov no início do século passado.

5.2 Modelo decisório de Markov

Markov percebeu que um agente inteligente, realizando ações sobre um meio instável, tomará decisões com base em informações de um ambiente passado, que não mais existe. Desta forma, o resultado da ação será apenas estatisticamente próximo do pretendido. Esta afirmação se justifica pela tendência, que o meio apresenta, a sofrer alterações gradativas, assim, os resultados das ações tendem a se aproximar do pretendido.

Suponha agora que um agente precisa atingir um objetivo por uma série de ações. Pelo método proposto por Markov, o agente analisa o meio após cada ação, medindo sua efetividade na aproximação do objetivo. A efetividade será traduzida numericamente em uma “recompensa”, positiva ou negativa conforme o agente se aproxima ou se distancia do objetivo. A somatória das recompensas



servirá de parâmetro para avaliar a eficiência da sequência de decisões, chamada por Markov de **política** de decisão do agente.

O modelo decisório de Markov ou MDP se baseia na suposição de que o resultado de uma ação futura pode ser estimado a partir de um conjunto finito de ações/recompensas (Norwig, 2013, p. 498).

Suponha agora que possamos, antes mesmo de tomar uma decisão, avaliar, com base em dados do ambiente, qual a recompensa de cada ação futura a partir do momento atual. Vamos imaginar um agente que deve se deslocar da localização atual (**estado atual**) para um determinado ponto em uma sala. Podemos saber, em função de experiências anteriores de deslocamento, onde estavam os obstáculos. Assim, para avaliar cada possível percurso, podemos utilizar as “recompensas” conhecidas anteriormente. Com este cálculo é possível estimar o que Markov chamou de **utilidade** de cada política ou sequência de ações futuras.

A utilidade **U** do estado atual para uma dada política será então (Norwig, 2013, p. 566):

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \dots$$

Buscando a maior **U** (utilidade ótima) será possível determinar o melhor caminho entre o estado atual e o objetivo. O equacionamento matemático desta maximização foi proposto por Bellman e nos fornece possíveis soluções para este problema de otimização.

Bellman buscou o conceito de valor da ação, **Q**, ou seja, avalia quão boa é uma determinada ação a partir do estado atual. Voltando a nossa suposição do agente decidindo como deslocar-se na sala. Estando ele em um determinado estado ou localização, poderá tomar, hipoteticamente, quatro ações: retornar de onde veio, seguir em frente, virar à direita ou à esquerda. Para cada uma destas possíveis ações podemos estimar **U**, conhecida a política. A melhor **U** resultará no melhor **Q** (Pellegrini; Wainer, 2007, p. 139).

De fato, a estimativa do melhor **Q**, como acabamos de concluir, depende da política (ou sequência de ações do agente). Desta forma, seria possível determinar qual a melhor política, pela seleção do maior **Q**, entre os melhores **Q** de cada política. Poderíamos generalizar ainda mais tomando o melhor **Q** para todos os estados possíveis, a este valor denominaremos **Q_{max}**. Encontrado **Q_{max}**,



a política ligada a ele será a melhor política e, portanto, descreverá as melhores decisões estatísticas de todo percurso.

O cálculo de Q_{max} por métodos matemáticos é extremamente custoso, embora possível, na maioria dos casos. Um problema cuja matemática de modelagem é complexa em demasia, ou mesmo desconhecida, é um problema ideal para tratamento por RNAs.

5.3 Deep Q-Learning Network

As redes neurais podem ser pensadas para auxiliar em processos de aprendizagem por reforço, como recurso para a estimativa da política ótima via maximização do valor da ação, Q_{max} .

As equações de Belman, de alta complexidade matemática, podem ser transformadas em uma equação recorrente para o cálculo de Q_{max} . Não será nosso objetivo demonstrar os processos de conversão, neste estudo. Utilizaremos, para nossa discussão, a equação proposta por Sutton (2018) para cálculo recorrente de Q_{max} em sistemas estocásticos temporais (Sutton, 2018):

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s' a') - Q(s, a))$$

Onde s designa um estado qualquer e s' seu sucessor, a representa a ação tomada no estado s e a' a ação em s' , α e γ são constantes de aprendizagem e r a recompensa obtida em s .

Veja a forte relação que esta equação tem com o cálculo recorrente de w_i por métodos supervisionados, com uso de gradiente descendente, que vimos em (2.1). Se aceitarmos essa relação, o uso de redes neurais se torna uma opção natural para o cálculo da política ótima. A partir daí, o método de treinamento não mais é importante, mas sim o resultado, que será a política ótima para o agente, no ambiente genérico estocástico temporal.

Para determinar a arquitetura desta RNA, vamos imaginar que o agente pode conhecer previamente o ambiente com base em uma imagem óptica. Aceitando esta hipótese, podemos convolucionar a imagem, extraíndo suas características por DL e então maximizar a função valor do estado, oferecendo, posteriormente, o resultado para uma segunda rede mais rasa.



Este processo é excessivamente complexo para que o tratemos neste ponto de nosso estudo, mas é possível perceber a aplicabilidade de DL, como ferramenta, para aprendizagem por reforço.

FINALIZANDO

Neste capítulo, evoluímos a compreensão das redes neurais artificiais, entendendo seu treinamento, especialmente aquele não supervisionado, importante para processos de *deep learning*. Estudamos também a aplicabilidade de redes neurais profundas no cálculo da política ótima, em algoritmos de aprendizagem por reforço. Concluímos, assim, a base de nosso estudo e estamos agora prontos para conhecer aplicações de aprendizagem profunda em nossas outras discussões



REFERÊNCIAS

DI BIASE, F. Sistemas auto-organizadores físicos, biológicos, sociais e empresariais. **International Journal of Knowledge Engineering and Management** (IJKEM), v. 2, n. 2, p. 123-146, 2013.

GÉRON, A. **Mãos à obra**: aprendizado de máquina com scikit-learn, keras, and tensorflow. 2. ed. Rio de Janeiro: Alta Books, 2021.

HAYKIN, S. **Redes neurais**. Porto Alegre: Grupo A, 2011.

NORVIG, P. **Inteligência artificial**. 3. ed. São Paulo: Grupo GEN, 2013.

PELLEGRINI, J.; WAINER, J. Processos de decisão de Markov: um tutorial. **Revista de Informática Teórica e Aplicada**, v. 14, n. 2, p. 133-179, 2007.

SUTTON, R. S.; BARTO, A. G. **Reinforcement learning**: an introduction. Massachusetts, EUA: MIT press, 2018.